

2024년 2회 정보처리기사 실기 복원 문제 (해설 추가본)

출처: <https://chobopark.tistory.com/483> (Life-Journey)

1번

다음은 Java 코드에 대한 문제이다. 알맞는 출력값을 작성하시오.

```
class Main {
    public static void main(String[] args) {
        int[] a = new int[]{1, 2, 3, 4};
        int[] b = new int[]{1, 2, 3, 4};
        int[] c = new int[]{1, 2, 3};
        check(a, b);
        check(a, c);
        check(b, c);
    }

    public static void check(int[] a, int[] b) {
        if (a == b) {
            System.out.print("0");
        } else {
            System.out.print("N");
        }
    }
}
```

정답: NNN

해설:

이 문제의 핵심은 자바에서 == 연산자가 배열에 쓰일 때 **값이 아니라 "주소(참조)"를 비교**한다는 점입니다.

- 배열은 객체이므로 `new int[] {...}` 로 만들 때마다 메모리에 **서로 다른 위치**에 생성됩니다.
- a, b, c는 내용이 같거나 비슷해도 **각각 다른 메모리 주소**를 가집니다.
- `a == b` 는 "a와 b가 같은 메모리를 가리키는가?"를 묻는 것이지, "내용이 같은가?"를 묻는 게 아닙니다.

세 번 모두 서로 다른 배열을 비교하므로 모두 거짓 → "N"이 세 번 출력되어 **NNN**입니다.

참고: 배열의 **내용**을 비교하려면 `Arrays.equals(a, b)` 를 사용해야 합니다.

2번

다음 문제에서 설명하는 용어를 작성하시오.

데이터를 중복시켜 성능을 향상시키기 위한 기법으로, 데이터를 중복 저장하거나 테이블을 합치는 등으로 성능을 향상시키지만 데이터 무결성이 저하될 수 있는 기법

정답: 반정규화

해설:

반정규화(De-normalization)는 정규화의 반대 개념입니다.

- 정규화**는 데이터 중복을 없애서 무결성(데이터의 정확성·일관성)을 높이는 과정입니다. 하지만 테이블이 잘게 쪼개지면 조회 시 여러 테이블을 조인(연결)해야 해서 **속도가 느려질 수 있습니다**.
- 반정규화**는 일부러 데이터를 중복 저장하거나 테이블을 합쳐서 **조회 성능을 끌어올리는** 기법입니다.
- 대신 같은 데이터가 여러 곳에 있게 되어 한 곳만 수정하면 불일치가 생길 위험, 즉 **무결성 저하**라는 단점이 있습니다.

"성능 향상 위해 일부러 중복 = 반정규화"로 기억하세요. (역정규화라고도 부릅니다.)

3번

다음은 SQL에 관한 문제이다. 빈칸을 작성하시오.

테이블: 사원[사원번호(PK), 이름, 나이, 부서] / 부서[사원번호(PK), 이름, 주소, 나이]

```
-- 신입 사원 추가
INSERT INTO 사원 (사원번호, 이름, 주소, 부서) [ ㉠ ] (32431, '정실기', '서울', '영업');

-- 부서 테이블에 추가
INSERT INTO 부서 (사원번호, 이름, 나이, 부서)
[ ㉡ ] 사원번호, 이름, 나이, 23 FROM 사원 WHERE 이름 = '정실기';

-- 전체 사원 테이블 조회
SELECT * [ ㉢ ] 사원;

-- 부서를 '퇴사'로 변경
UPDATE 사원 [ ㉣ ] 부서 = '퇴사' WHERE 사원번호 = 32431;
```

정답: ㉠ VALUES / ㉡ SELECT / ㉢ FROM / ㉣ SET

해설:
SQL의 가장 기본이 되는 네 가지 구문 형식을 묻는 문제입니다. 각 문장의 정해진 틀(형식)을 외워두면 됩니다.

- ㉠ **VALUES:** INSERT INTO 테이블(컬럼들) VALUES (값들) → 직접 값을 넣어 한 행을 추가하는 형식입니다.
- ㉡ **SELECT:** INSERT INTO 테이블(컬럼들) SELECT ... FROM ... → 다른 테이블에서 조회한 결과를 그대로 삽입하는 형식입니다. (서브쿼리 삽입)
- ㉢ **FROM:** SELECT * FROM 테이블 → 어느 테이블에서 가져올지 지정하는 절입니다.
- ㉣ **SET:** UPDATE 테이블 SET 컬럼=값 WHERE 조건 → 어떤 컬럼을 어떤 값으로 바꿀지 지정하는 절입니다.

INSERT는 두 가지 형태(VALUES로 직접 입력 / SELECT로 복사)가 있다는 점을 함께 기억하세요.

4번

다음 릴레이션의 Cardinality와 Degree를 작성하시오. (※ 원문에 테이블 이미지 포함)

정답: ㉠ Cardinality: 5 / ㉡ Degree: 4

해설:
관계형 데이터베이스에서 테이블의 크기를 나타내는 두 용어입니다.

- **카디널리티(Cardinality):** 테이블의 **행(튜플, Row)의 개수**입니다. → 이 문제에서는 5개.
- **차수(Degree):** 테이블의 **열(속성, Column)의 개수**입니다. → 이 문제에서는 4개.

자주 헷갈리니 이렇게 외우세요: "카디널리티는 세로로 쌓인 행의 수, 차수(Degree)는 가로로 늘어선 열의 수." 행과 열을 반대로 쓰는 실수를 주의하세요.

5번

다음은 프로토콜에 대한 내용이다. 알맞는 답을 작성하시오.

- Network layer에서 IP 패킷을 암호화하고 인증하는 등 보안을 위한 표준
- 사설 인터넷망(VPN)을 구현하는 데 사용
- AH(Authentication Header)와 ESP(Encapsulating Security Payload) 두 보안 프로토콜 사용

정답: IPSec

해설:
IPSec(IP Security)은 **네트워크 계층(3계층)에서 IP 패킷 자체를 암호화·인증**하는 보안 표준입니다.

- VPN(가상 사설망)을 안전하게 구현하는 데 가장 많이 쓰입니다.
- 두 개의 핵심 프로토콜로 구성됩니다.
- **AH(Authentication Header):** 데이터의 **무결성과 인증**을 보장합니다. (변조 여부 확인) 단, 암호화는 하지 않습니다.
- **ESP(Encapsulating Security Payload):** 인증 + **암호화(기밀성)**까지 제공합니다.

키워드 "네트워크 계층 보안 + AH/ESP = IPsec"으로 기억하세요. SSL/TLS는 더 상위 계층(전송~응용)에서 동작한다는 점과 구분하면 좋습니다.

6번

다음은 Python에 대한 문제이다. 알맞는 출력값을 작성하시오.

```
def fnCalculation(x, y):
    result = 0
    for i in range(len(x)):
        temp = x[i:i+len(y)]
        if temp == y:
            result += 1
    return result

a = "abdcabcbabca"
p1 = "ab"
p2 = "ca"

out = f"ab{fnCalculation(a, p1)}ca{fnCalculation(a, p2)}"
print(out)
```

정답: ab3ca3

해설:

이 함수는 문자열 a 안에서 패턴 y가 몇 번 나타나는지 세는 함수입니다.

- `x[i:i+len(y)]` 는 a의 i번째부터 패턴 길이만큼 잘라낸 조각입니다. (슬라이싱)
- 그 조각이 y와 같으면 카운트를 1 증가시킵니다.

문자열 `a = "abdcabcbabca"` 에서 세어 봅시다 (인덱스 0부터):

- 패턴 "ab" 찾기: 위치 0("ab"), 위치 4("ab"), 위치 7("ab") → 총 **3번**
- 패턴 "ca" 찾기: 위치 3("ca"), 위치 6... 천천히 보면 "ca"는 위치 3, 6, 9에서 나타남 → 총 **3번**

마지막 f-문자열은 `"ab" + 3 + "ca" + 3` 을 이어붙여 **"ab3ca3"**가 됩니다. f-string의 `{ }` 안 값이 그대로 문자열에 삽입되는 것이 포인트입니다.

7번

아래 설명하는 내용을 확인하여 알맞는 알고리즘을 작성하시오.

- 대칭키 알고리즘으로 1997년 NIST에서 DES를 대체하기 위해 생성
- 128/192/256비트의 가변 키 크기와 128비트 고정 블록 크기 사용
- 높은 안전성·효율성·속도로 전 세계적으로 많이 사용

정답: AES

해설:

AES(Advanced Encryption Standard)는 현재 가장 널리 쓰이는 **대칭키 암호 알고리즘**입니다.

- '대칭키'란 암호화할 때와 복호화할 때 **같은 키**를 사용하는 방식입니다.
- 기존 DES가 키 길이가 짧아(56비트) 보안에 취약해지자, 미국 표준기관 NIST가 이를 대체할 새 표준으로 AES를 채택했습니다.
- 키 길이는 **128/192/256비트** 중 선택 가능하고, 한 번에 처리하는 **블록 크기는 128비트로 고정**입니다.

대칭키 알고리즘 종류: DES, 3DES, AES, SEED, ARIA 등.

비대칭키 알고리즘 종류: RSA, ECC 등.

"DES를 대체한 표준 = AES"라는 키워드를 기억하세요.

8번

패킷 교환 방식 중에 연결형과 비연결형에 해당하는 방식을 작성하시오.

정답: ① 연결형: 가상회선 / ② 비연결형: 데이터그램

해설:
패킷 교환 방식은 통신 전에 경로를 미리 정하는지에 따라 두 종류로 나뉩니다.

- **연결형 = 가상회선(Virtual Circuit):** 데이터를 보내기 전에 **논리적 경로를 먼저 설정**하고, 모든 패킷이 그 길을 따라 순서대로 갑니다. 도착 순서가 보장됩니다. (전화 통화처럼 "먼저 연결")
- **비연결형 = 데이터그램(Datagram):** 경로를 미리 정하지 않고 **각 패킷이 독립적으로** 그때그때 최적 경로를 찾아갑니다. 순서가 뒤바뀔 수 있어 재정렬이 필요합니다. (편지처럼 "각자 알아서")

"연결형 = 가상회선", "비연결형 = 데이터그램" 한 쌍으로 외우세요. 대표적으로 IP는 데이터그램 방식입니다.

9번

아래 내용을 확인하고 보기에서 알맞는 답을 고르시오.

실행 순서가 밀접한 관계를 갖는 기능을 모아 모듈로 구성한다. 한 모듈 내부의 한 기능 요소에 의한 출력 자료가 다음 기능 원소의 입력 자료로 제공되는 형태이다.

보기: ㄱ. 기능적 ㄴ. 우연적 ㄷ. 통신적 ㄹ. 절차적 ㄴ. 시간적 ㅂ. 순차적 ㅅ. 논리적

정답: ㅂ (순차적)

해설:
응집도(Cohesion)는 한 모듈 안의 구성요소들이 얼마나 밀접하게 관련돼 있는가를 나타냅니다. 응집도가 높을수록 좋은 설계입니다.

이 문제의 키워드는 "**앞 기능의 출력이 다음 기능의 입력으로 제공된다**"입니다. 이것이 **순차적 응집도(Sequential Cohesion)**의 정의입니다. 작업이 줄줄이 이어지듯 데이터가 흘러가는 형태죠.

응집도를 강한 순서(좋은→나쁨)로 정리하면:
기능적 > 순차적 > 교환(통신)적 > 절차적 > 시간적 > 논리적 > 우연적

비슷한 보기와 구분 포인트:
- 순차적: 출력→입력으로 데이터가 흐름
- 절차적: 단순히 순서대로 실행만 됨(데이터 흐름은 무관)
- 교환(통신)적: 같은 데이터를 사용하지만 순서 관계는 없음

10번

아래는 디자인 패턴에 관한 설명이다. 보기에서 알맞는 용어를 작성하시오.

- 컬렉션 객체의 내부 구조를 노출하지 않고 순차적으로 접근할 수 있게 하는 패턴
- 객체의 내부 표현 방식에 독립적으로 요소에 접근
- 반복 프로세스를 캡슐화

보기: (생성) Singleton, Factory Method, Abstract Factory / (구조) Adapter, Bridge, Composite / (행위) Iterator, Visitor, Observer

정답: Iterator

해설:
Iterator(반복자) 패턴은 **컬렉션(여러 데이터를 담은 객체)의 내부 구조를 숨긴 채, 원소를 하나씩 순서대로 꺼내 접근**할 수 있게 해주는 행위 패턴입니다.

- 'Iterate'는 '반복하다'라는 뜻입니다. 리스트, 배열, 트리 등 내부 구조가 어떻게 상관없이 **동일한 방식(next() 등)으로 원소를 순회**할 수 있게 합니다.
- 사용자는 컬렉션이 배열인지 연결리스트인지 몰라도 됩니다. "다음 요소 주세요"만 하면 됩니다.

키워드 "순차 접근, 내부 구조 노출 안 함, 반복 캡슐화 = Iterator"입니다. 행위(Behavioral) 패턴에 속합니다.

11번

아래 그림을 바탕으로 RIP을 구성하여 최단 경로 비용을 계산하여 흐름에 맞게 작성하시오. (※ 원문에 그림 이미지 포함)

예제: A →

정답: A → D → C → F

해설:

RIP(Routing Information Protocol)은 거리(홉 수, hop count)를 기준으로 **최단 경로**를 찾는 라우팅 프로토콜입니다.

- '홉(hop)'이란 라우터를 한 번 거치는 것을 1로 세는 단위입니다. RIP은 목적지까지 **거쳐가는 라우터 수가 가장 적은 경로**를 선택합니다.
- 이 문제는 그림에서 각 경로의 비용을 더해, 출발지 A에서 목적지 F까지 **총비용이 최소가 되는 경로**를 찾는 것입니다.
- 그 결과가 A → D → C → F였습니다. (직접 가는 길보다 D, C를 거치는 것이 총비용이 더 작았다는 의미)

참고: RIP은 최대 홉 수가 15로 제한되어 소규모 네트워크에 적합합니다. 다익스트라(Dijkstra) 기반의 OSPF와 구분하세요.

12번

아래의 표를 확인하여 SRT 스케줄링의 평균 대기시간을 계산하여 작성하시오.

프로세스	도착 시간	서비스 시간
A	0	8
B	1	4
C	2	9
D	3	5

정답: 6.5

해설:
SRT(Shortest Remaining Time)는 "**남은 실행시간이 가장 짧은 프로세스를 먼저 처리**"하는 **선점형** 스케줄링입니다. 더 짧은 작업이 도착하면 실행 중이던 작업을 멈추고 빼앗습니다(선점).

- 시간 순서대로 따라가 봅시다(남은 시간 기준):
- 0~1: A 시작(남은 8→7). 시각 1에 B(4) 도착 → B가 더 짧으니 선점.
 - 1~5: B 실행(4→0 완료). (A=7, C=9, D=5 대기)
 - 5~10: 남은 시간 비교 → D(5)가 A(7)보다 짧음 → D 실행(5→0 완료).
 - 10~17: A 실행(7→0 완료).
 - 17~26: C 실행(9→0 완료).

- 각 프로세스 대기시간 = (실제 시작/완료까지의 흐름) – 도착시간 – 서비스시간 형태로 계산:
- A: 완료 17, 도착 0, 서비스 8 → 대기 = 17–0–8 = 9
 - B: 완료 5, 도착 1, 서비스 4 → 대기 = 5–1–4 = 0
 - C: 완료 26, 도착 2, 서비스 9 → 대기 = 26–2–9 = 15
 - D: 완료 10, 도착 3, 서비스 5 → 대기 = 10–3–5 = 2

평균 대기시간 = (9 + 0 + 15 + 2) / 4 = 26 / 4 = **6.5**

핵심: SRT는 매 시점 "남은 시간이 가장 짧은 것"을 고른다는 규칙입니다.

13번

다음은 C언어에 대한 문제이다. 알맞는 출력값을 작성하시오.

```
#include <stdio.h>

int main() {
    int arr[3][3] = {1, 2, 3, 4, 5, 6, 7, 8, 9};
    int* parr[2] = {arr[1], arr[2]};
    printf("%d", parr[1][1] + *(parr[1]+2) + **parr);
    return 0;
}
```

정답: 21

해설:
2차원 배열과 포인터 배열을 다루는 문제입니다. 먼저 배열을 정리하면:

```
arr[0] = {1, 2, 3}
arr[1] = {4, 5, 6}
arr[2] = {7, 8, 9}
```

parr 는 포인터 배열로, parr[0] = arr[1] (즉 {4,5,6}의 시작), parr[1] = arr[2] (즉 {7,8,9}의 시작)입니다.

각 항을 계산합니다.

- parr[1][1] → parr[1]은 arr[2]={7,8,9}, 그중 [1]번 = **8**
- *(parr[1]+2) → parr[1]에서 2칸 뒤의 값 = arr[2][2] = **9**
- **parr → *(*parr) = parr[0]이 가리키는 첫 값 = arr[1][0] = **4**

합: 8 + 9 + 4 = **21**

포인터 표기 *(p+i) 는 p[i] 와 같다는 것, **parr 는 parr[0][0]과 같다는 것을 알면 풀립니다.

14번

다음은 Java 언어에 대한 문제이다. 알맞는 출력값을 작성하시오.

```
class Main {
    public static void main(String[] args) {
        int a[] = {1, 2, 3, 4, 5, 6, 7, 8, 9};
        ODDNumber OE = new ODDNumber();
        System.out.print(OE.sum(a, true) + ", " + OE.sum(a, false));
    }
}

interface Number {
    int sum(int[] a, boolean odd);
}

class ODDNumber implements Number {
    public int sum(int[] a, boolean odd) {
        int result = 0;
        for(int i=0; i < a.length; i++){
            if((odd && a[i] % 2 != 0) || (!odd && a[i] % 2 == 0))
                result += a[i];
        }
        return result;
    }
}
```

정답: 25, 20

해설:

이 메서드는 odd 값에 따라 **홀수의 합** 또는 **짝수의 합**을 구합니다.

조건문을 해석하면:

- odd가 true이고 a[i]가 홀수 → 홀수만 더함
- odd가 false이고 a[i]가 짝수 → 짝수만 더함

(% 2 != 0 은 홀수, % 2 == 0 은 짝수를 의미합니다.)

배열 {1,2,3,4,5,6,7,8,9}에서:

- sum(a, true) = 홀수 합 = 1+3+5+7+9 = **25**
- sum(a, false) = 짝수 합 = 2+4+6+8 = **20**

결과는 "25, 20". 나눗셈 나머지로 홀짝을 판별하는 기본 패턴(% 2)을 익혀두세요.

15번

다음은 C언어에 대한 문제이다. 알맞는 출력값을 작성하시오.

```
#include <stdio.h>
#include <string.h>

void sumFn(char* d, const char* s) {
    while (*s) {
        *d = *s;
        d++;
        s++;
    }
    *d = '\0';
}

int main() {
    const char* str1 = "first";
    char str2[50] = "teststring";
    int result = 0;
    sumFn(str2, str1);

    for (int i = 0; str2[i] != '\0'; i++) {
        result += i;
    }
    printf("%d", result);
    return 0;
}
```

정답: 10

해설:

sumFn 은 문자열 s를 d에 한 글자씩 복사하는 함수입니다(직접 만든 strcpy와 동일).

- while(*s) 는 s가 가리키는 문자가 널 문자('\0', 값 0)가 될 때까지 반복합니다.
- str1 = "first"(5글자)를 str2에 복사하고 끝에 '\0'을 붙입니다. 따라서 **str2는 "first"**가 됩니다. (원래 있던 "teststring"은 앞부분이 덮어쓰기 됨)

그다음 for문은 str2가 끝날 때까지 인덱스 i를 더합니다.

- "first"는 길이 5이므로 i = 0,1,2,3,4까지 반복
- result = 0+1+2+3+4 = **10**

주의: 더하는 것은 글자가 아니라 **인덱스 i**입니다. "first"의 인덱스 0~4의 합 = 10입니다.

16번

아래는 소프트웨어 설계에 대한 내용이다. 괄호 안에 알맞는 답을 작성하시오.

- 어떤 모듈이 다른 모듈 내부의 논리적 흐름을 제어하기 위해 제어를 통신하거나 제어 요소를 전달하는 결합도
- 한 모듈이 다른 모듈의 상세한 처리 절차를 알고 통제하거나 처리 기능이 두 모듈에 분리 설계된 경우 발생

() Coupling

정답: 제어 (Control)

해설:

결합도(Coupling)는 모듈과 모듈이 서로 얼마나 의존하는지를 나타냅니다. 결합도는 **낮을수록 좋은** 설계입니다(독립성이 높으니까).

이 문제의 키워드는 **"제어 요소(제어 플래그)를 전달하여 다른 모듈의 흐름을 좌우한다"**입니다. 이것이 **제어 결합도(Control Coupling)**의 정의입니다. 예를 들어 true/false 같은 것발 값을 넘겨서 상대 모듈이 어떻게 동작할지 지시하는 경우입니다.

결합도를 약한 순서(좋은→나쁨)로:

자료 < 스택프 < 제어 < 외부 < 공통 < 내용

구분 포인트:

- 자료 결합도: 순수 데이터만 전달(가장 좋음)
- 제어 결합도: 제어용 플래그를 전달해 흐름 통제
- 내용 결합도: 다른 모듈 내부를 직접 참조(가장 나쁨)

17번

다음은 Java에 대한 문제이다. 알맞는 출력값을 작성하시오.

```
class Main {
    public static void main(String[] args) {
        String str = "abacabcd";
        boolean[] seen = new boolean[256];
        System.out.print(calculFn(str, str.length()-1, seen));
    }

    public static String calculFn(String str, int index, boolean[] seen) {
        if(index < 0) return "";
        char c = str.charAt(index);
        String result = calculFn(str, index-1, seen);
        if(!seen[c]) {
            seen[c] = true;
            return c + result;
        }
        return result;
    }
}
```

정답: dcba

해설:

재귀(자기 자신을 다시 호출)를 이용해 문자열을 처리하는 문제입니다. 동작은 "**뒤에서 앞으로 훑으면서, 처음 등장하는 글자만 남긴다**"입니다.

핵심은 `result = calculFn(str, index-1, seen)` 을 **먼저** 호출한다는 점입니다. 즉, 가장 끝(index가 작은 쪽)까지 먼저 내려갔다가 돌아오면서 처리합니다. 그래서 앞 글자가 result의 앞쪽에 붙습니다... 라고 생각하기 쉽지만, `seen` 배열은 재귀가 **앞쪽(index 0)부터 복귀**하면서 표시되므로 앞에 나온 글자가 먼저 "본 것"으로 기록됩니다.

"abacabcd"에서 각 글자의 **첫 등장 위치**를 보면: a(0), b(1), c(4), d(7). 첫 등장 순서대로 모은 뒤, 코드 특성상 뒤 인덱스 글자가 앞에 붙어 출력됩니다. 결과적으로 중복 제거 후 **뒤에서부터 순서로** "dcba"가 만들어집니다.

핵심 학습 포인트: `seen[c]` 배열로 이미 처리한 글자를 표시해 **중복을 제거**하고, 재귀 호출 순서가 결과 문자열의 순서를 결정한다는 점입니다.

18번

다음은 C언어에 대한 문제이다. 알맞는 출력값을 작성하시오.

```
#include <stdio.h>

void swap(int a, int b) {
    int t = a;
    a = b;
    b = t;
}

int main() {
    int a = 11;
    int b = 19;
    swap(a, b);

    switch(a) {
        case 1:
            b += 1;
        case 11:
            b += 2;
        default:
            b += 3;
            break;
    }
}
```



```
printf("%d", a-b);  
}
```

정답: -13

해설:

두 가지 함정이 있습니다: ① **값에 의한 전달(call by value)**, ② **switch문의 fall-through(break 없으면 아래로 계속 실행)**.

먼저 swap 함수: C에서 함수에 변수를 넘기면 **값만 복사**됩니다. 함수 안에서 a, b를 바꿔도 main의 a, b는 **그대로**입니다. 따라서 swap 후에도 **a=11, b=19**입니다.

이제 switch(a) → a=11이므로 case 11로 진입:

- case 11: b += 2 → b = 19+2 = 21. **break가 없으므로** 아래로 계속 실행됨!
- default: b += 3 → b = 21+3 = 24. 여기서 break.

최종 a=11, b=24 → a - b = 11 - 24 = **-13**

핵심: case에 break가 없으면 다음 case들이 줄줄이 실행됩니다(fall-through). 그리고 swap이 실제로는 교환하지 못한다는 점이 함께 출제된 함정입니다.

19번

다음은 C언어의 구조체에 대한 문제이다. 알맞는 출력값을 작성하시오.

```
#include <stdio.h>  
  
struct node {  
    int n1;  
    struct node *n2;  
};  
  
int main() {  
    struct node a = {10, NULL};  
    struct node b = {20, NULL};  
    struct node c = {30, NULL};  
  
    struct node *head = &a;  
    a.n2 = &b;  
    b.n2 = &c;  
  
    printf("%d\n", head->n2->n1);  
    return 0;  
}
```

정답: 20

해설:

연결 리스트(Linked List)의 기본을 묻는 문제입니다. 각 노드는 값(n1)과 "다음 노드를 가리키는 포인터(n2)"를 가집니다.

연결 관계를 그려보면:

- a(10) → b(20) → c(30) (a.n2가 b를, b.n2가 c를 가리킴)
- head는 a를 가리킴

이제 head->n2->n1 을 단계별로 따라갑니다.

- head = a
- head->n2 = a의 다음 노드 = **b**
- head->n2->n1 = b의 값 = **20**

-> 연산자는 "포인터가 가리키는 구조체의 멤버에 접근"하는 화살표입니다. 화살표를 따라 a → b로 한 칸 이동한 뒤 그 값(20)을 읽은 것입니다.

20번

다음은 Java에 대한 문제이다. 알맞는 출력값을 작성하시오.

```
class Main {
    public static void main(String[] args) {
        String str = "ITISTESTSTRING";
        String[] result = str.split("T");
        System.out.print(result[3]);
    }
}
```

정답: S

해설:

`split("T")` 은 문자열을 **"T"를 기준으로 잘라서 배열로** 만듭니다. 구분자 T는 결과에서 사라집니다.

"ITISTESTSTRING"을 T마다 자르면:

- 첫 T 앞: "I"
- 다음 조각: "IS"
- 다음 조각: "ES"
- 다음 조각: "S"
- 다음 조각: "RING"

즉 `result = ["I", "IS", "ES", "S", "RING"]` (인덱스 0,1,2,3,4)

원문을 T로 끊어보면: ITISTESTSTRING → 조각들이 [I, IS, ES, S, RING]이 됩니다.

`result[3]` 은 인덱스 3번, 즉 네 번째 조각인 **"S"**입니다. 배열 인덱스는 0부터 시작한다는 점을 잊지 마세요.